

FinPlanner Renewals & Enhancements - Design Decisions

Document Overview

This document outlines the key design decisions made during the refactoring and extension of FinPlanner to support comprehensive renewals/planned obligations tracking, global command bar, and enhanced SMS inbox functionality.

1. Core Architecture Decision: Extend Bills vs New Renewals Model

Decision: Extend the existing Bills module

Context

The task required implementing a first-class “Renewals/Planned Obligations” module to track recurring obligations like insurance, school fees, taxes, etc. Two approaches were considered:

1. **Option A:** Create a new, separate `Renewal` model with its own tables and API routes
2. **Option B:** Extend the existing `Bill` and `BillInstance` models

Analysis

Option A: New Renewals Module

Pros:

- Clean separation of concerns
- Can design schema specifically for renewals
- No impact on existing Bills functionality

Cons:

- Significant code duplication (similar frequency logic, payment tracking)
- Need to maintain two parallel systems
- Complex aggregation for “Upcoming Items” (need to query two separate tables)
- Larger codebase to maintain
- More database models and migrations

Option B: Extend Bills Module CHOSEN

Pros:

- Leverages existing infrastructure (`Bill`, `BillInstance` models already exist)
- Frequency enums (`ONE_TIME`, `MONTHLY`, `QUARTERLY`, `HALF_YEARLY`, `YEARLY`) match requirements exactly
- `BillInstance` provides payment history structure
- Less code duplication
- Simpler “Upcoming Items” aggregation (single table query)
- Faster implementation
- Natural semantic fit: renewals ARE a type of recurring bill

Cons:

- Bills are conceptually recurring payments, might not semantically fit all “obligation” types
- Need to add fields to existing models (requires migration)
- Slightly less flexibility in schema design

Final Decision

Extend Bills Module for the following reasons:

1. **Perfect Frequency Match:** The existing `BillFrequency` enum already includes all required frequencies
2. **Payment History:** `BillInstance` provides exactly the payment tracking structure needed
3. **DRY Principle:** Avoids duplicating frequency logic and payment workflows
4. **Maintainability:** Simpler to maintain one cohesive system
5. **Natural Fit:** Semantically, renewals (insurance, subscriptions, etc.) are bills
6. **Faster Time to Market:** Leverages 80% of existing infrastructure

Implementation Approach

1. **Schema Extension:** Add `provider`, `policyNumber`, `reminderDays` to `Bill` model
2. **Logic Enhancement:** Implement automatic `nextDueDate` rolling when marking paid
3. **Status Computation:** Add utility functions for overdue/due-soon status
4. **UI Enhancement:** Update forms and displays with new fields
5. **Backwards Compatible:** Existing bills continue to work without changes

2. Automatic Date Rolling Logic

Decision: Roll `nextDueDate` automatically when marking a bill as paid

Context

When a user marks a recurring bill/renewal as paid, the system needs to automatically calculate the next due date based on the frequency.

Implementation**Core Function**

```
function calculateNextDueDate(currentDueDate: Date, frequency: BillFrequency): Date
```

Edge Cases Handled

1. **Month-End Dates (Jan 31 → Feb 28/29)**
 - Problem: Setting a bill due on Jan 31 and rolling monthly should handle February correctly
 - Solution: If day changes after month addition (e.g., Jan 31 → Mar 3), set to last day of intended month
 - Example: Jan 31 → Feb 28 (non-leap) or Feb 29 (leap year)
2. **Leap Years (Feb 29 handling)**
 - Problem: A bill due on Feb 29, 2024 (leap year) rolls to 2025 (non-leap year)

- Solution: Adjust to Feb 28 when next year isn't a leap year
- Example: Feb 29, 2024 → Feb 28, 2025 → Feb 28, 2026 → Feb 28, 2027 → Feb 29, 2028

3. Year Rollovers

- Problem: December bills rolling to January of next year
- Solution: Use JavaScript Date's built-in month/year handling
- Example: Dec 15, 2025 + 1 month = Jan 15, 2026

4. ONE_TIME Bills

- Problem: One-time bills shouldn't roll
- Solution: Return the same date without modification
- Rationale: ONE_TIME frequency indicates a single obligation

Testing Strategy

Created comprehensive test suite (`lib/__tests__/bill-utils.test.ts`) covering:

- All frequency types (WEEKLY, MONTHLY, QUARTERLY, HALF_YEARLY, YEARLY, ONE_TIME)
- Edge cases (month-end, leap years, year rollover)
- 20+ test cases ensuring correctness

API Integration

The automatic rolling is triggered in `/api/bills/[id]/payment` POST endpoint:

```
if (bill.frequency !== 'ONE_TIME') {
  const newNextDueDate = calculateNextDueDate(new Date(bill.nextDueDate), bill.frequency)
  await prisma.bill.update({
    where: { id: billId },
    data: { nextDueDate: newNextDueDate }
  })
}
```

3. Reminder System Design

Decision: Store as comma-separated string, compute dynamically

Context

Users need configurable reminders for upcoming bills/renewals (e.g., 30 days, 7 days, 1 day before due).

Schema Design

```
reminderDays: String @default("30,7,1")
```

Rationale:

- **Flexibility:** Users can customize reminder thresholds
- **Simplicity:** Easy to parse and validate
- **Storage Efficiency:** Single string field vs array of integers
- **Backwards Compatible:** Default value works for all bills

Parsing Function

```
function parseReminderDays(reminderDaysStr: string): number[] {
  return reminderDaysStr
    .split(',')
    .map(d => parseInt(d.trim(), 10))
    .filter(d => !isNaN(d) && d > 0)
    .sort((a, b) => b - a) // Sort descending
}
```

Reminder Logic

Reminders are computed dynamically based on:

1. **Days Until Due:** Calculate `daysUntilDue = targetDate - today`
2. **Threshold Check:** Show reminder if `daysUntilDue <= any threshold`
3. **Status Indicator:** Set `reminderStatus` (30-day, 7-day, 1-day)

UI Representation

- **Badges:** Color-coded badges for different reminder levels
- **Highlights:** Row highlighting for due-soon items
- **Visual Only:** No email/SMS notifications (as per requirements)

4. Transaction Creation on Bill Payment

Decision: Automatically create transaction when marking bill as paid (default)

Context

When a user marks a bill as paid, should it automatically create a transaction record?

Options Considered

Option A: Always create transaction (no option to disable)

- Pro: Keeps records in sync
- Con: No flexibility if user has already recorded transaction

Option B: Never create transaction

- Pro: User has full control
- Con: Extra manual work, records can get out of sync

Option C: Create by default, allow opt-out ☒ **CHOSEN**

- Pro: Convenience + flexibility
- Con: Slightly more complex API

Implementation

```
// In payment endpoint
const { createTransaction = true } = body // Default to true

if (createTransaction) {
  const transaction = await prisma.transaction.create({
    data: {
      amount: amount || bill.amount,
      type: 'EXPENSE',
      description: `${bill.name}${bill.provider ? ` - ${bill.provider}` : ''}`,
      merchant: bill.provider || bill.name,
      source: 'BILL',
      categoryId: bill.categoryId,
      userId: userId
    }
  })

  billInstance.transactionId = transaction.id
}
```

Benefits

1. **Consistency:** Transaction records stay in sync with bill payments
2. **Analytics:** Accurate month-to-date expense tracking
3. **Traceability:** Link between bill payment and transaction
4. **Flexibility:** Can disable if user prefers manual entry

5. Global Command Bar - LLM Integration

Decision: Use Abacus.AI LLM with fallback pattern matching Context

Users want to quickly add transactions/renewals using natural language from anywhere in the app.

Architecture

Primary: Abacus.AI LLM

```
const client = new AbacusAI.ApiClient()
const response = await client.create_chat_llm_response({
  messages: [
    { is_user: false, text: systemPrompt },
    { is_user: true, text: userInput }
  ],
  llm_name: 'GPT_4_TURBO',
  temperature: 0.3
})
```

Rationale:

- **Accuracy:** LLM handles complex, varied natural language better than patterns
- **Flexibility:** Can understand context and inference
- **Maintainability:** Less regex maintenance vs pattern-based parsing

Fallback: Pattern Matching

```
function fallbackParse(input: string) {
  // Extract amount using regex
  // Detect keywords (spent, paid, renewal, etc.)
  // Return structured data
}
```

Rationale:

- **Reliability:** Works if LLM API is down or slow
- **Cost:** Saves API calls for simple, clear inputs
- **Offline:** Could work without internet (future consideration)

User Experience Flow

1. User presses **Cmd/Ctrl + K** or clicks floating action button
2. User types natural language command
3. System parses using LLM (with fallback)
4. Shows parsed result with confidence level
5. User confirms/edits category selection
6. System creates transaction/renewal

Confidence Levels

- **High:** All key fields extracted clearly → Auto-suggest confirmation
- **Medium:** Some inference needed → Show for review
- **Low:** Ambiguous input → Show error, suggest examples

6. SMS Inbox Enhancement Strategy

Decision: Batch processing with LLM, review/approve workflow

Context

Existing SMS parser uses pattern matching for Indian banks. Enhancement needed for:

- Multiple SMS at once
- Better accuracy with LLM
- Review/approval workflow
- SMS traceability

Architecture

Existing (`transaction-parser.ts`)

- Pattern-based parsing
- Bank-specific regex patterns
- Single SMS at a time
- Immediate creation (no review)

Enhanced (`/api/transactions/parse-batch`)

- LLM-based parsing with Abacus.AI
- Generic format support (not India-specific)

- Batch processing (multiple SMS)
- Review/approve workflow
- Raw SMS storage

Implementation Approach

1. **Input:** User pastes multiple SMS messages (separated by newlines)
2. **Parsing:** Each SMS sent to LLM for parsing
3. **Aggregation:** Results collected with confidence scores
4. **Review UI:** Show parsed transactions in table
5. **User Action:** Approve, edit, or reject each transaction
6. **Creation:** Only approved transactions are created
7. **Storage:** Raw SMS text stored in `transaction.rawMessage` field

LLM Prompt Strategy

```
const systemPrompt = `Parse this SMS into a transaction.
Extract: amount, type (debit/credit), merchant, account, transaction ID, date, balance.
Return JSON with confidence level.`
```

Benefits

1. **Accuracy:** LLM handles varied SMS formats better
2. **Control:** User reviews before transactions are created
3. **Traceability:** Raw SMS stored for future reference
4. **Batch Efficiency:** Process 5-10 SMS at once
5. **Error Recovery:** User can edit incorrect parses

7. Dashboard Month-to-Date Stats

Decision: Current calendar month (Jan 1-31), not rolling 30 days

Context

User requirement: “Month-to-date income/expenses/net (ensure correctness for current calendar month)”

Implementation

```
function getCurrentMonthBounds(): { start: Date; end: Date } {
  const now = new Date()
  const start = new Date(now.getFullYear(), now.getMonth(), 1)
  const end = new Date(now.getFullYear(), now.getMonth() + 1, 0, 23, 59, 59, 999)
  return { start, end }
}
```

Rationale

1. **User Expectation:** When users think “this month”, they mean calendar month
2. **Standard Practice:** Most finance apps use calendar months

3. **Clear Boundaries:** Jan 1-31, Feb 1-28/29, etc. are intuitive
4. **Comparison:** Easier to compare month-over-month

Dashboard Widgets

1. **Income This Month:** Sum of all INCOME transactions in current calendar month
 2. **Expenses This Month:** Sum of all EXPENSE transactions in current calendar month
 3. **Net This Month:** Income - Expenses
 4. **Visual Indicators:** Green for positive net, red for negative
-

8. Upcoming Obligations Aggregation

Decision: Single query with 30/60-day filter, combine bills and status

Context

Dashboard needs to show upcoming bills/renewals for the next 30 or 60 days.

Implementation

```
const upcomingItems = await prisma.bill.findMany({
  where: {
    userId: userId,
    isActive: true,
    nextDueDate: {
      gte: new Date(),
      lte: new Date(Date.now() + daysAhead * 24 * 60 * 60 * 1000)
    }
  },
  include: {
    category: true,
    instances: {
      where: { status: 'PENDING' },
      orderBy: { dueDate: 'desc' }
    }
  },
  orderBy: { nextDueDate: 'asc' }
})
```

Benefits of Single Model

- **Simple Query:** One table query vs joining two tables
- **Consistent UI:** All items displayed uniformly
- **Efficient:** No need for complex aggregation logic
- **Maintainable:** Single source of truth

Filtering and Sorting

1. **Filter:** Only active bills, unpaid instances, within date range
 2. **Sort:** By due date ascending (soonest first)
 3. **Status Indicators:** Color-coded by overdue/due-soon/upcoming
 4. **Reminder Badges:** Show reminder threshold (30-day, 7-day, 1-day)
-

9. Currency and Date Formats

Decision: INR (₹) as default currency, Indian date formats

Context

User specified: "Currency: INR (Indian Rupees), Date format: Indian date formats"

Implementation

Currency Formatting

```
export function formatCurrency(amount: number): string {
  return new Intl.NumberFormat('en-IN', {
    style: 'currency',
    currency: 'INR',
    minimumFractionDigits: 0,
    maximumFractionDigits: 2
  }).format(amount)
}
// Example: formatCurrency(1000) => "₹1,000"
```

Date Formatting

```
date.toLocaleDateString('en-IN', {
  day: 'numeric',
  month: 'short',
  year: 'numeric'
})
// Example: "15 Jan, 2026"
```

Rationale

1. **User Preference:** Explicitly requested by user
 2. **Localization:** Aligns with Indian financial practices
 3. **Readability:** Lakhs/crores formatting (future consideration)
-

10. Empty States and Onboarding

Decision: Setup checklist for new users, hero section for upcoming obligations

Context

New users need guidance, empty states should encourage action.

Approach

Empty State Components

1. **No Bills/Renewals:** Show call-to-action to create first bill
2. **No Upcoming Obligations:** Show benefits of tracking obligations
3. **No Transactions:** Guide to add first transaction or paste SMS

Setup Checklist

```
const setupSteps = [
  { id: 1, title: 'Add your first renewal', completed: billsCount > 0 },
  { id: 2, title: 'Paste SMS for auto-import', completed: smsTransactionsCount > 0 },
  { id: 3, title: 'Set up categories', completed: categoriesCount > 5 }
]
```

Hero Section

- **Location:** Top of dashboard
- **Content:** “Upcoming Obligations” with visual timeline
- **Action:** Quick add renewal button
- **Empty State:** Encouraging message with examples

Benefits

1. **User Activation:** Guides users to core features
2. **Reduced Confusion:** Clear next steps
3. **Feature Discovery:** Highlights key capabilities
4. **Engagement:** Encourages initial data entry

11. Testing Strategy

Decision: Unit tests for business logic, manual E2E for UI workflows

Context

Need to ensure date rolling logic correctness and overall system reliability.

Test Coverage

Unit Tests (`__tests__/bill-utils.test.ts`)

✓ Implemented

- Date rolling for all frequencies
- Edge cases (month-end, leap years)
- Status computation
- Reminder logic
- Amount calculations

API Tests

✗ Not Implemented (recommended for future)

- CRUD operations for bills
- Payment workflow
- Command parsing accuracy
- SMS batch parsing

E2E Tests

✗ Manual Testing Required

- Create renewal → See in upcoming → Mark paid → Verify date rolls
- SMS inbox flow

- Command bar with natural language
- Dashboard stats accuracy

Rationale

1. **Critical Logic:** Date rolling has high complexity → needs unit tests
2. **UI Workflows:** Best validated manually initially
3. **Time Constraints:** Prioritize core logic tests first
4. **Future:** Add E2E tests with Playwright/Cypress later

12. Migration Strategy

Decision: Additive migrations only, backwards compatible

Context

Existing production database has bills data. New fields need to be added without breaking existing functionality.

Migration Approach

```
model Bill {
  // Existing fields (unchanged)
  id          String      @id @default(cuid())
  name        String
  amount      Float
  frequency   BillFrequency
  // ... other existing fields

  // New fields (all optional or with defaults)
  provider    String? // NULL for existing bills
  policyNumber String? // NULL for existing bills
  reminderDays String @default("30,7,1") // Default for existing bills
}

model BillInstance {
  // Existing fields (unchanged)
  id          String      @id @default(cuid())
  // ... other existing fields

  // New field (optional)
  referenceNumber String? // NULL for existing instances
}
```

Benefits

1. **No Data Loss:** Existing bills continue to work
2. **Gradual Adoption:** Users can add provider/policy info as needed
3. **Default Values:** New fields have sensible defaults
4. **Rollback Safety:** Can rollback without data corruption

Summary of Key Decisions

Decision Area	Choice Made	Rationale
Architecture	Extend Bills module	Leverage existing infrastructure, avoid duplication
Date Rolling	Automatic on payment	User convenience, handles edge cases correctly
Reminders	Comma-separated string, computed dynamically	Flexibility, simplicity, visual indicators only
Transaction Creation	Auto-create by default (opt-out)	Keeps records in sync, provides flexibility
Command Bar	LLM with fallback	Best accuracy, reliability, natural language support
SMS Inbox	Batch + LLM + Review workflow	Better accuracy, user control, traceability
Dashboard Period	Current calendar month	User expectation, standard practice
Currency	INR (₹) with Indian formatting	User requirement, localization
Testing	Unit tests for logic, manual for UI	Prioritize critical logic, time-efficient
Migration	Additive, backwards compatible	Safe, no data loss, gradual adoption

Future Considerations

Potential Enhancements

1. **Email/SMS Notifications:** Currently visual only, could add optional notifications
2. **Multi-Currency:** Support for multiple currencies and conversions
3. **Bank Integration:** Direct API connections with banks (requires regulatory compliance)
4. **Recurring Transactions:** Auto-create transactions for recurring bills
5. **Payment Methods:** Track which payment method was used (card, UPI, cash)
6. **Document Attachments:** Attach policy documents, receipts to renewals
7. **Family Sharing:** Multi-user access for shared bills/renewals
8. **Budgeting:** Set budgets based on recurring obligations
9. **AI Recommendations:** Suggest optimal payment dates based on cash flow

Technical Debt

1. **API Tests:** Add comprehensive API endpoint tests
 2. **E2E Tests:** Implement Playwright/Cypress test suite
 3. **Performance:** Add database indexes for frequent queries
 4. **Caching:** Implement Redis caching for dashboard stats
 5. **Real-time:** WebSocket updates for live dashboard
 6. **Mobile:** React Native app for mobile access
-

Conclusion

These design decisions prioritize **user convenience**, **data integrity**, and **maintainability** while delivering the required features within the project constraints. The approach of extending the Bills module rather than creating a new system proved to be the most efficient and maintainable solution, allowing us to deliver a comprehensive renewals/planned obligations system with automatic date rolling, LLM-powered natural language entry, and enhanced SMS processing capabilities.

All decisions are documented, tested (where critical), and designed for future extensibility without breaking existing functionality.